



**DURBAN UNIVERSITY OF TECHNOLOGY**  
**INYUVESI YASETHEKWINI YEZOBUCHWEPHESHE**

**FACULTY OF ACCOUNTING AND INFORMATICS**

**DEPARTMENT OF INFORMATION TECHNOLOGY**

**2026**  
**FEBRUARY SPECIAL EXAMINATION**

**INSTRUCTIONAL PROGRAMME :** B INFORMATION & COMMUNICATION TECHNOLOGY

**INSTRUCTIONAL OFFERING :** INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

**PAPER NUMBER :** 1

**SUBJECT CODE/S :** IPRT301

**DATE :** 12 FEBRUARY 2026

**DURATION :** 3 HOURS

**TIME :** 09H00 - 12H00

**TOTAL MARKS :** 100

**NUMBER OF PAGES :** 8 (INCLUDING COVER PAGE)

**EXAMINER/S :** BOAMAH-ABU, CHARLES (PhD)

**MODERATOR/S :** RUSHIL RAGHAVJEE (PhD)

---

**INSTRUCTIONS/REQUIREMENTS:-**

1. This examination is closed book
2. Answer all questions
3. Where stated "briefly", please answer as briefly as possible
4. Ensure your handwriting is readable

***Do not turn the page until permission is given***

1. In each of the following, indicate whether the statement is true or false. Write **T** (for **true**) or **F** (for **false**). (5)

1.1. Inheritance applies ONLY to classes.

1.2. A class can inherit an interface.

1.3. In method override, the appropriate methods are selected at run time.

1.4. An abstract class which consists of **only** constants and abstract methods can be converted to an interface.

1.5. Any class that inherits an abstract class must also be declared abstract.

2. State and briefly describe one advantage of an interface over abstract class. (2)

3. Briefly answer the following.

3.1. What is method overload? (2)

3.2. Explain why the code below is wrong. (2)

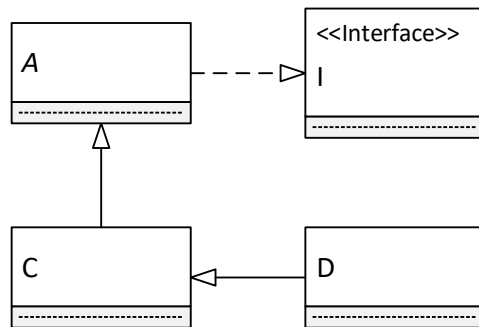
```
...  
...  
public class Square {  
    public int a(int x) {  
        return x*x;  
    };  
    public double a(int x) {  
        return x*x;  
    }  
}
```

4. Briefly answer the following.

4.1. What is polymorphism? (2)

4.2. A class **Student** extends a class **Person**. Use this to explain polymorphism. (2)

5. The figure below shows **I** is an interface; **A** is an abstract class; and **C** and **D** are concrete classes. In each of the following, state whether the statement is **valid** or **invalid**. Explain your answer. (10)

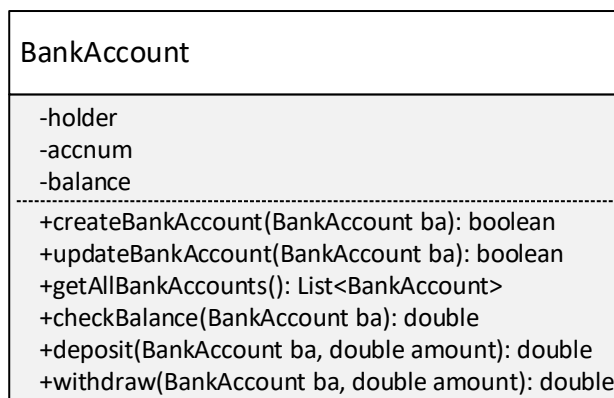


1. I ia=new I();
2. I ib=new A();
3. A aa=new C();
4. I ic=new C();
5. D da=new C();

## Question 2 (Design patterns)

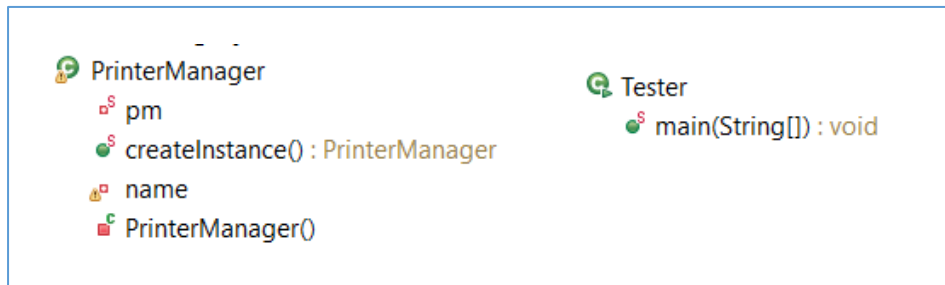
25 marks

1. Mention one advantage of design patterns in software development. (2)
2. What are the main uses of the following patterns? (4)
  - a. Template
  - b. DAO
3. Below is the class **BankAccount**.



- 3.1. Briefly explain why the class **BankAccount** is poorly designed. (2)
- 3.2. How could the problem in 3.1 be resolved? (2)

4. A school has only one printer, which is shared by all users. The singleton class **PrinterManager** creates **only** one instance of the printer, which is shared by all users. Below shows the structure of **PrinterManager** and **Tester** (a class which creates an instance of **PrinterManager**).



- 4.1. Complete the code below for the singleton class **PrinterManager** (use **Lazy initialization**). (9)

```
...
...

public class PrintManager{
    _____
    _____
    _____
    _____
    _____
    _____
    _____
    _____
}
```

- 4.2. Explain why you cannot call the constructor in **Tester** to create an instance of **PrinterManager**. (2)

- 4.3. In the **main** method (in the class **Tester** below):

- 4.3.1. Write the missing line of code to create an instance of **PrinterManager**. (2)

- 4.3.2. Explain the importance of the keyword **synchronized** the **main** method above. (2)

```
package singleton;

public class Tester {

    public synchronized static void main(String[] args) {
        //create an instance of PrinterManager
        _____
        System.out.println(p1);
    }
}
```

1. Mention one advantage and one disadvantage of XML. (4)
2. . Below is part of the data on books sold at NLM Bookstore. Each book has **isbn**, **name** and **price**.

|               |                           |     |
|---------------|---------------------------|-----|
| 001-002-002-1 | Java for dummies          | 150 |
| 001-002-002-2 | Design patterns           | 200 |
| 001-002-002-3 | Data mapping and exchange | 250 |

The data is stored as XML in the file **book.xml**. The root node is **Library**, which contains **book** nodes. Each **book** node has an attribute **isbn** (the **isbn** of the book) and two elements: **name** and **price** (of the book).

- 2.1. Complete the XML code (below) for the three book in **book.xml**. (7)

```
<?xml version="1.0" encoding="UTF-8"?>
```

---

---

---

---

---

---

---

---

---

---

---

---

- 2.2. Complete the **DTD** code (below) in a file **book.dtd** for the XML in **book.xml**. (7)

```
<?xml version="1.0" encoding="UTF-8"?>
```

---

---

---

---

---

2.3. The data in **book.xml** is to be displayed (as shown below) in a web page.

| ISBN          | Name                      | Price |
|---------------|---------------------------|-------|
| 001-002-002-1 | Java for dummies          | 150   |
| 001-002-002-2 | Design patterns           | 200   |
| 001-002-002-3 | Data mapping and exchange | 250   |

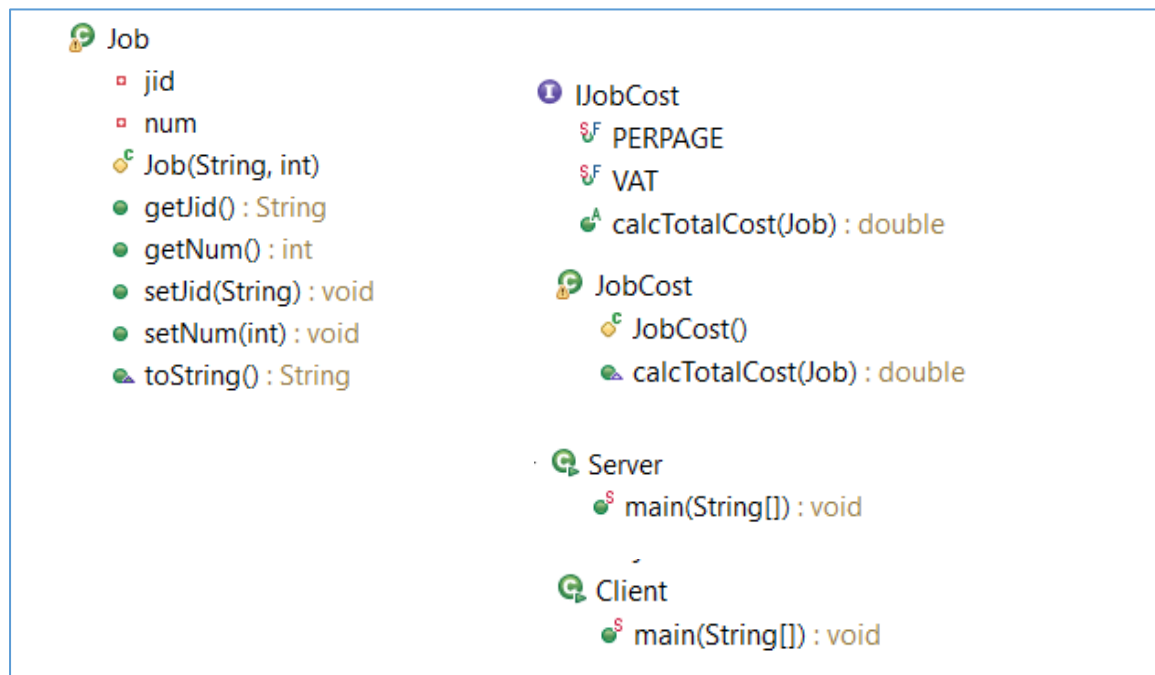
Below is the XSL code, in a file **book.xsl**. Complete the missing parts (labelled **A**, **B**, **C** and **D**). Write only the letter followed by your answer. (7)

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

    <xsl:template match="/Library">
        <title>Library books</title>
        <table border="1">
            <tr>
                <th>ISBN</th>
                <th>Name</th>
                <th>Price</th>
            </tr>
            <xsl:for-each select= _____ A _____>
                <tr>
                    <td><_____ B _____>/></td>
                    <td><_____ C _____>/> </td>
                    <td>_____ D _____ price"/> </td>
                </tr>
            </xsl:for-each>
        </table>
    </xsl:template>
</xsl:stylesheet>
```

## Question 4 RMI

1. What is the main disadvantage of RMI? (2)
2. Sharon Printing provides printing services to clients. A printing job consists of the job id (**jid**) and number of pages (**num**). You are required to write an RMI application. The figure below shows the class **Job**, which represents a printing job. **IJobCost** is a remote interface; the class **JobCost** implements the remote interface. The class **Server** registers and binds an instance of **JobCost**. The class **Client** calls the remote methods of **JobCost**.



- The class **Job** has **jid** (job id) and **num** (number of pages) as well as a **constructor**; **get**, **set** and **toString** methods
- The interface **IJobCost** has **PERPAGE** (the printing cost per page, a constant equal to R1.00); **VAT** (the VAT chargeable, a constant equal to 15%) and a method **calcTotalCost**
- **JobCost** implements **IJobCost**. **JobCost** is a non-parameterised constructor and the method **calcTotalCost** returns the total cost of a printing job. This is calculated as the total printing cost (i.e., number of pages X the printing cost per page) and VAT payable (calculated as 15% of the printing cost).

Use the information above to answer **Questions 1 to 4** below.

1. Write the code for the remote interface **IJobCost**. (6)

```
...
...
_____ {
    _____
    _____
    _____
    _____
}
```

2. Complete the code for the class **JobCost** (which implements **IJobCost**) as follows:

2.1. Write the class header. (3)

2.2. Write the constructor. (2)

3. Complete the class **Server** (below) to register and bind an instance (called **jc**) of **JobCost** with the reference “**job**”, using port **2025**. The **main** method declares all exceptions. (6)

```
...
...
...
public class Server {
    public static void main(String[] args) _____ {
        _____
        _____
        _____
    }
}
```

4. Complete the class **Client** (below) as follows:

4.1. Complete the code, which returns the reference “**job**” registered by the **Server** and assigns it to **ijc**. (3)

4.2. Complete the code that uses **ijc** to call the remote method **calcTotalCost** with an instance of **Job** (**j**) and displays the result on the screen. (3)

```
package rmi;
import java.rmi.Naming;

public class Client {
    public static void main(String[] args) throws Exception {
        IJobCost ijc=_____
        Job j=new Job("001", 100);
        System.out.println(j);
        System.out.println("Total cost of job="+_____);
    }
}
```